# Coding & Solution

| | |
|---|---|
| **Date** | 1 July 2026 |
| **Team ID** | xxxxxx |
| **Project Name** | Emotion Detection & Learning Support Engine |
| **Maximum Marks** | 5 Marks |

## Coding & Solution

The **Emotion Detection & Learning Support Engine** is developed using **Python, Streamlit, Machine Learning, Natural Language Processing (NLP), and Generative AI** to detect users' emotions from text and provide personalized learning support. The system utilizes a fine-tuned **BERT emotion classification model** for accurate emotion prediction and integrates the **Google Gemini API** to deliver intelligent guidance and interactive responses. The project includes text preprocessing, model loading, emotion classification, personalized learning resource recommendations, and an intuitive web-based interface. The code follows a modular architecture with reusable components, proper error handling, and clean coding practices to ensure reliability, scalability, and maintainability.

## Solution Summary

| Field | Details |
|---|---|
| **Repository Link / URL** | https://github.com/venkateshmalakala/Emotion-Detection-Learning-Support-Engine |
| **Programming Language(s)** | Python, HTML, CSS, JavaScript |
| **Framework(s) Used** | Streamlit, Transformers (Hugging Face), PyTorch, Pandas, NumPy, NLTK, Google Gemini API |
| **Key Features Implemented** | Emotion detection using BERT, AI-powered chatbot, personalized learning recommendations, text preprocessing, interactive Streamlit web interface, real-time emotion prediction |

| | |
|---|---|
| **Pending / Incomplete Features** | User authentication, database integration, multilingual support, voice emotion recognition, cloud deployment, progress tracking dashboard |
| **Setup / Run Instructions** | Install dependencies using **pip install -r requirements.txt**, activate the virtual environment (if applicable), run **streamlit run app.py**, and open the local Streamlit URL (typically **http://localhost:8501** |

## Code Quality Checklist

| S.No | Criteria | Status (Yes / No) |
|---|---|---|
| 1 | Code is modular and organized into functions/modules | Yes |
| 2 | Meaningful variable and function names are used | Yes |
| 3 | Code includes comments/documentation where necessary | Yes |
| 4 | Error handling is implemented for critical operations | Yes |
| 5 | The application runs without critical errors | Yes |
| 6 | Code is committed to a version control repository (GitHub) | Yes |

## Additional Notes / Comments:

The project follows a clean and modular coding structure with proper documentation, meaningful variable names, and error handling. It successfully detects user emotions, provides personalized learning recommendations, and offers AI-powered guidance through the Gemini API. The application is easy to maintain and can be enhanced with features such as user authentication, database integration, multilingual support, and cloud deployment in future versions.